```
AWSTemplateFormatVersion: '2010-09-09'
Description: 'Fixyachips Global v4.0: COMPLETE STACK - OAC, Origin Shield, WAF, AI Suite,
Signup API, Paywall-Ready'

# ============================================================
# WHAT'S IN THIS FILE (v4.0 COMPLETE)
# ============================================================
# 1.  EmpireOAC            - Origin Access Control (S3 lock)
# 2.  WebBodyguard         - WAF firewall (restored from v3.4)
# 3.  WebsiteRouterFunction - CloudFront edge URL router
# 4.  StreamingVideoBucket  - Secured S3 bucket (CORS locked)
# 5.  BucketPolicy         - Correct ARN handshake (FIXED)
# 6.  WebsiteDistribution  - CloudFront + Origin Shield + WAF
# 7.  StudentTable         - DynamoDB (DeletionPolicy: Retain)
# 8.  FixyachipsMasterRole - Scoped IAM (FIXED - not wildcard)
# 9.  SignupWorker         - Lambda pre-signup + SES email
# 10. SignupWorkerLogGroup  - CloudWatch logs for Lambda
# 11. SignupAPI            - API Gateway REST API
# 12. ApiResource          - /signup route
# 13. ApiMethod            - POST handler
# 14. ApiDeployment        - Deploys API to 'prod' stage
# 15. LambdaPermission     - Lets API Gateway invoke Lambda
# 16. FixyachipsUserPool   - Cognito user pool
# 17. FixyachipsUserPoolClient - Cognito web client
# ============================================================

Parameters:
  AdminEmail:
    Type: String
    Default: 'info@fixyachips.com'

Resources:

  # ============================================================
  # 1. THE LOCK (Origin Access Control)
  # Tells CloudFront to sign every S3 request with SigV4.
  # This means your S3 bucket stays 100% private.
  # ============================================================
  EmpireOAC:
    Type: 'AWS::CloudFront::OriginAccessControl'
    Properties:
      OriginAccessControlConfig:
        Name: !Sub 'Empire-OAC-${AWS::StackName}'
        OriginAccessControlOriginType: s3
        SigningBehavior: always
        SigningProtocol: sigv4

  # ============================================================
  # 2. THE BODYGUARD (WAF - Web Application Firewall)
```

```
# Restored from v3.4. Protects against bots, SQL injection,
# XSS attacks, and common exploits at the CloudFront edge.
# Cost: ~$5/month base + $0.60 per million requests.
# MUST be in us-east-1 for CloudFront scope.
# ============================================================
WebBodyguard:
  Type: 'AWS::WAFv2::WebACL'
  Properties:
    Name: !Sub 'FixyachipsShield-${AWS::StackName}'
    Scope: CLOUDFRONT
    DefaultAction: { Allow: {} }
    VisibilityConfig:
      SampledRequestsEnabled: true
      CloudWatchMetricsEnabled: true
      MetricName: FixyachipsWAF
    Rules:
      - Name: AWS-AWSManagedRulesCommonRuleSet
        Priority: 0
        Statement:
          ManagedRuleGroupStatement:
            VendorName: AWS
            Name: AWSManagedRulesCommonRuleSet
        OverrideAction: { None: {} }
        VisibilityConfig:
          SampledRequestsEnabled: true
          CloudWatchMetricsEnabled: true
          MetricName: CommonRules
      - Name: AWS-AWSManagedRulesBotControlRuleSet
        Priority: 1
        Statement:
          ManagedRuleGroupStatement:
            VendorName: AWS
            Name: AWSManagedRulesBotControlRuleSet
            ManagedRuleGroupConfigs:
              - AWSManagedRulesBotControlRuleSet:
                  InspectionLevel: COMMON
        OverrideAction: { None: {} }
        VisibilityConfig:
          SampledRequestsEnabled: true
          CloudWatchMetricsEnabled: true
          MetricName: BotControl

# ============================================================
# 3. THE EDGE ROUTER (CloudFront Function)
# Handles clean URLs - turns /about into /about.html
# Runs at the edge in every CloudFront location worldwide.
# ============================================================
WebsiteRouterFunction:
  Type: 'AWS::CloudFront::Function'
```

```
  Properties:
    Name: !Sub 'FixRouter-${AWS::StackName}'
    AutoPublish: true
    FunctionConfig:
      Comment: 'Fixes .html paths for clean URLs'
      Runtime: 'cloudfront-js-1.0'
    FunctionCode: |
      function handler(event) {
        var request = event.request;
        var uri = request.uri;
        if (uri.endsWith('/')) { request.uri += 'index.html'; }
        else if (!uri.includes('.')) { request.uri += '.html'; }
        return request;
      }

# ============================================================
# 4. THE STORAGE (S3 Bucket)
# Fully private - no public access whatsoever.
# CORS locked to fixyachips.com only (FIXED from v3.9).
# ============================================================
StreamingVideoBucket:
  Type: 'AWS::S3::Bucket'
  Properties:
    BucketName: !Sub 'fixyachips-platform-${AWS::AccountId}'
    PublicAccessBlockConfiguration:
      BlockPublicAcls: true
      BlockPublicPolicy: true
      IgnorePublicAcls: true
      RestrictPublicBuckets: true
    CorsConfiguration:
      CorsRules:
        - AllowedHeaders: ['*']
          AllowedMethods: [GET, POST, PUT]
          AllowedOrigins:
            - 'https://fixyachips.com'
            - 'https://www.fixyachips.com'

# ============================================================
# 5. THE HANDSHAKE (Bucket Policy)
# Only allows CloudFront (via OAC) to read from S3.
# ARN fixed from v3.9 (was 'arn:aws:partition' - not real).
# ============================================================
BucketPolicy:
  Type: 'AWS::S3::BucketPolicy'
  Properties:
    Bucket: !Ref StreamingVideoBucket
    PolicyDocument:
      Version: '2012-10-17'
      Statement:
```

```
        - Effect: Allow
          Principal:
            Service: cloudfront.amazonaws.com
          Action: 's3:GetObject'
          Resource: !Sub 'arn:aws:s3:::${StreamingVideoBucket}/*'
          Condition:
            StringEquals:
              AWS:SourceArn: !Sub 'arn:aws:cloudfront::${AWS::AccountId}:distribution/$
{WebsiteDistribution}'

  # ============================================================
  # 6. THE GLOBAL WEBSITE (CloudFront Distribution)
  # PriceClass_All = all 400+ edge locations worldwide.
  # Origin Shield = extra caching layer, saves S3 costs.
  # WAF wired in via WebAclId.
  # ============================================================
  WebsiteDistribution:
    Type: 'AWS::CloudFront::Distribution'
    Properties:
      DistributionConfig:
        Enabled: true
        PriceClass: PriceClass_All
        DefaultRootObject: index.html
        # WAF connected to CloudFront distribution
        WebACLId: !GetAtt WebBodyguard.Arn
        Origins:
          - Id: S3Origin
            DomainName: !GetAtt StreamingVideoBucket.RegionalDomainName
            # S3OriginConfig required as placeholder even when using OAC
            S3OriginConfig:
              OriginAccessIdentity: ""
            OriginAccessControlId: !GetAtt EmpireOAC.Id
            # Origin Shield: extra caching layer between CloudFront and S3
            OriginShield:
              Enabled: true
              OriginShieldRegion: us-east-1
        DefaultCacheBehavior:
          TargetOriginId: S3Origin
          ViewerProtocolPolicy: redirect-to-https
          Compress: true
          ForwardedValues:
            QueryString: false
          FunctionAssociations:
            - EventType: viewer-request
              FunctionARN: !GetAtt WebsiteRouterFunction.FunctionMetadata.FunctionARN

  # ============================================================
  # 7. THE DATABASE (DynamoDB)
  # Stores every student record.
```

```
# DeletionPolicy: Retain = data survives even if stack deleted.
# PAY_PER_REQUEST = $0 cost until you have real students.
# ============================================================
StudentTable:
  Type: 'AWS::DynamoDB::Table'
  DeletionPolicy: Retain
  Properties:
    TableName: !Sub 'FixyachipsStudents-${AWS::StackName}'
    AttributeDefinitions:
      - AttributeName: StudentID
        AttributeType: S
    KeySchema:
      - AttributeName: StudentID
        KeyType: HASH
    BillingMode: PAY_PER_REQUEST

# ============================================================
# 8. THE BRAINS (IAM Role)
# Used by all Lambda functions.
# FIXED: Scoped to specific resources only (not wildcard *).
# ============================================================
FixyachipsMasterRole:
  Type: 'AWS::IAM::Role'
  Properties:
    AssumeRolePolicyDocument:
      Version: '2012-10-17'
      Statement:
        - Effect: Allow
          Principal: { Service: [lambda.amazonaws.com] }
          Action: ['sts:AssumeRole']
    ManagedPolicyArns:
      - 'arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole'
    Policies:
      - PolicyName: FixyachipsAIPowerhouse
        PolicyDocument:
          Version: '2012-10-17'
          Statement:
            # S3: scoped to your bucket only
            - Effect: Allow
              Action:
                - 's3:GetObject'
                - 's3:PutObject'
                - 's3:DeleteObject'
                - 's3:ListBucket'
              Resource:
                - !Sub 'arn:aws:s3:::${StreamingVideoBucket}'
                - !Sub 'arn:aws:s3:::${StreamingVideoBucket}/*'
            # DynamoDB: scoped to your student table only
            - Effect: Allow
```

```
            Action:
              - 'dynamodb:GetItem'
              - 'dynamodb:PutItem'
              - 'dynamodb:UpdateItem'
              - 'dynamodb:DeleteItem'
              - 'dynamodb:Query'
              - 'dynamodb:Scan'
            Resource: !GetAtt StudentTable.Arn
          # AI/ML: these services require * resource (AWS limitation)
          - Effect: Allow
            Action:
              - 'ses:SendEmail'
              - 'bedrock:*'
              - 'transcribe:*'
              - 'translate:*'
              - 'polly:*'
            Resource: '*'

# ============================================================
# 9. THE SIGNUP WORKER (Lambda Function)
# Restored from v3.4. Handles pre-launch waitlist signups.
# Flow: Frontend form -> API Gateway -> This Lambda ->
#       DynamoDB (stores lead) + SES (sends welcome email)
# ============================================================
SignupWorker:
  Type: 'AWS::Lambda::Function'
  Properties:
    FunctionName: !Sub 'FixyachipsSignupWorker-${AWS::StackName}'
    Handler: index.lambda_handler
    Runtime: python3.12
    Role: !GetAtt FixyachipsMasterRole.Arn
    Timeout: 30
    Environment:
      Variables:
        TABLE_NAME: !Ref StudentTable
        ADMIN_EMAIL: !Ref AdminEmail
    Code:
      ZipFile: |
        import json, boto3, uuid, os
        dynamodb = boto3.resource('dynamodb')
        ses = boto3.client('ses')
        table = dynamodb.Table(os.environ['TABLE_NAME'])

        def lambda_handler(event, context):

            if event.get('httpMethod') == 'OPTIONS':
                return {
                    'statusCode': 200,
                    'headers': {
```

```
                'Access-Control-Allow-Origin': 'https://fixyachips.com',
                'Access-Control-Allow-Headers': 'Content-Type',
                'Access-Control-Allow-Methods': 'POST, OPTIONS'
            },
            'body': ''
        }
    try:
        body = json.loads(event['body'])
        email = body.get('email', '').strip().lower()
        name = body.get('name', '').strip()

        if not email or not name:
            return {
                'statusCode': 400,
                'headers': {'Access-Control-Allow-Origin': 'https://fixyachips.com'},
                'body': json.dumps({'error': 'Name and email are required'})
            }


        table.put_item(Item={
            'StudentID': str(uuid.uuid4()),
            'Name': name,
            'Email': email,
            'Status': 'WAITLIST',
            'Source': body.get('source', 'website'),
            'Industry': body.get('industry', 'unknown')
        })


        ses.send_email(
            Source=os.environ['ADMIN_EMAIL'],
            Destination={'ToAddresses': [email]},
            Message={
                'Subject': {'Data': 'Welcome to the No-Drill Revolution - Fixyachips Academy'},
                'Body': {'Text': {'Data': 'Hello ' + name + ',\n\nYou are now on the waitlist for
Fixyachips Global Academy.\n\nYou are about to learn the same trade secret No-Drill windshield repair
methodology passed down since 1972 - personally taught to Joshua Cane by the pioneers themselves.\
n\nThis is not just a course. This is a business in a box.\n\nWe will notify you the moment enrollment
opens.\n\nJoshua Cane\nFounder, Fixyachips Global Academy\nCustodian of the 1972 No-Drill
Methodology'}}
            }
        )

        ses.send_email(
            Source=os.environ['ADMIN_EMAIL'],
            Destination={'ToAddresses': [os.environ['ADMIN_EMAIL']]},
            Message={
                'Subject': {'Data': f'New Waitlist Signup: {name}'},
```

```
                    'Body': {'Text': {'Data': f'New signup:\nName: {name}\nEmail: {email}\nIndustry:
{body.get("industry", "unknown")}'}}
                }
            )

            return {
                'statusCode': 200,
                'headers': {
                    'Access-Control-Allow-Origin': 'https://fixyachips.com',
                    'Content-Type': 'application/json'
                },
                'body': json.dumps({'status': 'success', 'message': 'Welcome to the revolution!'})
            }

        except Exception as e:
            print(f"ERROR: {str(e)}")
            return {
                'statusCode': 500,
                'headers': {'Access-Control-Allow-Origin': 'https://fixyachips.com'},
                'body': json.dumps({'error': 'Signup failed. Please try again.'})
            }

# ============================================================
# 10. CLOUDWATCH LOGS (Lambda Log Group)
# Gives your Lambda a named log group with 30-day retention.
# Without this logs pile up forever and cost money.
# ============================================================
SignupWorkerLogGroup:
  Type: 'AWS::Logs::LogGroup'
  Properties:
    LogGroupName: !Sub '/aws/lambda/FixyachipsSignupWorker-${AWS::StackName}'
    RetentionInDays: 30

# ============================================================
# 11. THE FRONT DOOR (API Gateway)
# Creates the REST API that your frontend form posts to.
# ============================================================
SignupAPI:
  Type: 'AWS::ApiGateway::RestApi'
  Properties:
    Name: !Sub 'FixyachipsAPI-${AWS::StackName}'
    Description: 'Fixyachips Signup and Course API'

# ============================================================
# 12. THE ROUTE (/signup resource)
# ============================================================
ApiResource:
  Type: 'AWS::ApiGateway::Resource'
  Properties:
```

```
      ParentId: !GetAtt SignupAPI.RootResourceId
      PathPart: 'signup'
      RestApiId: !Ref SignupAPI

  # ============================================================
  # 13. THE METHOD (POST handler)
  # ============================================================
  ApiMethod:
    Type: 'AWS::ApiGateway::Method'
    Properties:
      AuthorizationType: NONE
      HttpMethod: POST
      ResourceId: !Ref ApiResource
      RestApiId: !Ref SignupAPI
      Integration:
        IntegrationHttpMethod: POST
        Type: AWS_PROXY
        Uri: !Sub 'arn:aws:apigateway:${AWS::Region}:lambda:path/2015-03-31/functions/$
{SignupWorker.Arn}/invocations'

  # CORS OPTIONS method for browser preflight requests
  ApiOptionsMethod:
    Type: 'AWS::ApiGateway::Method'
    Properties:
      AuthorizationType: NONE
      HttpMethod: OPTIONS
      ResourceId: !Ref ApiResource
      RestApiId: !Ref SignupAPI
      Integration:
        IntegrationHttpMethod: POST
        Type: AWS_PROXY
        Uri: !Sub 'arn:aws:apigateway:${AWS::Region}:lambda:path/2015-03-31/functions/$
{SignupWorker.Arn}/invocations'

  # ============================================================
  # 14. THE DEPLOYMENT (publishes API to 'prod' stage)
  # ============================================================
  ApiDeployment:
    Type: 'AWS::ApiGateway::Deployment'
    DependsOn:
      - ApiMethod
      - ApiOptionsMethod
    Properties:
      RestApiId: !Ref SignupAPI
      StageName: prod

  # ============================================================
  # 15. LAMBDA PERMISSION
  # Lets API Gateway invoke the SignupWorker Lambda.
```

```
# ============================================================
LambdaPermission:
  Type: 'AWS::Lambda::Permission'
  Properties:
    Action: 'lambda:InvokeFunction'
    FunctionName: !Ref SignupWorker
    Principal: apigateway.amazonaws.com
    SourceArn: !Sub 'arn:aws:execute-api:${AWS::Region}:${AWS::AccountId}:${SignupAPI}/*'

# ============================================================
# 16. COGNITO USER POOL
# Handles student login/signup with email verification.
# Custom attributes: veteran_status, industry_segment.
# Strong password policy from v3.4 restored.
# ============================================================
FixyachipsUserPool:
  Type: 'AWS::Cognito::UserPool'
  Properties:
    UserPoolName: !Sub 'Fixyachips_Global_Users-${AWS::StackName}'
    UsernameAttributes: [email]
    AutoVerifiedAttributes: [email]
    Policies:
      PasswordPolicy:
        MinimumLength: 8
        RequireLowercase: true
        RequireNumbers: true
        RequireSymbols: true
        RequireUppercase: true
    Schema:
      - Name: email
        AttributeDataType: String
        Mutable: true
        Required: true
      - Name: veteran_status
        AttributeDataType: String
        Mutable: true
      - Name: industry_segment
        AttributeDataType: String
        Mutable: true

# ============================================================
# 17. COGNITO USER POOL CLIENT
# The web app client - no secret (public SPA client).
# ============================================================
FixyachipsUserPoolClient:
  Type: 'AWS::Cognito::UserPoolClient'
  Properties:
    ClientName: Fixyachips_Web_Portal
    UserPoolId: !Ref FixyachipsUserPool
```

```
      GenerateSecret: false
      ExplicitAuthFlows:
        - ALLOW_USER_PASSWORD_AUTH
        - ALLOW_REFRESH_TOKEN_AUTH
        - ALLOW_USER_SRP_AUTH

# ============================================================
# OUTPUTS - Values you'll need for your frontend code
# ============================================================
Outputs:
  WebsiteURL:
    Description: 'Your CloudFront website URL'
    Value: !GetAtt WebsiteDistribution.DomainName

  SignupApiUrl:
    Description: 'POST endpoint for waitlist signups'
    Value: !Sub 'https://${SignupAPI}.execute-api.${AWS::Region}.amazonaws.com/prod/signup'

  S3Bucket:
    Description: 'Your video/content storage bucket'
    Value: !Ref StreamingVideoBucket

  UserPoolId:
    Description: 'Cognito User Pool ID - needed for frontend auth'
    Value: !Ref FixyachipsUserPool

  UserPoolClientId:
    Description: 'Cognito Client ID - needed for frontend auth'
    Value: !Ref FixyachipsUserPoolClient

  WAFArn:
    Description: 'WAF Web ACL ARN'
    Value: !GetAtt WebBodyguard.Arn
```